

Deadline 기반 렌더팜 구축 및 자동화 시스템 설계

[redacted]

홍익대학교 영상애니메이션학부

Correspondence:

[redacted]

홍익대학교 영상애니메이션학부 / 컴퓨터공학과

Email: [redacted]

목차

1. 서론 (Introduction)

1.1 연구 배경

1.2 기존 실습실 렌더링 환경의 문제점

1.3 연구 목적

1.4 시스템 구축 목표 및 의의

2. 관련 연구 및 기술 배경 (Related Work & Background)

2.1 Render Farm 개념 및 스튜디오 사례

2.2 Deadline 렌더 매니지먼트 시스템 구조

2.3 MayaBatch / Arnold / Blender 렌더 아키텍처

2.4 NAS·네트워크 구조·파일 경로 설계 개념

2.5 PySide, MongoDB, Google Apps Script 기술 개요

3. 시스템 요구사항 분석 (Requirements Analysis)

3.1 기능적 요구사항

3.2 비기능적 요구사항

3.3 실습실 환경의 제약 조건

3.4 기술 선택 이유(Deadline / MongoDB / PySide 등)

4. 시스템 설계 (System Design) ← 지금 4장 + 일부 5.7에서 “개념만” 뽑기

4.1 전체 Workflow 설계

4.2 렌더팜 인프라 아키텍처 설계

4.3 렌더팜 인프라 아키텍처 설계

4.4 NAS 경로 정책 및 폴더 구조 설계

4.5 UI 설계 (기능 관점)

4.6 데이터 흐름 설계 (Data Flow)

4.7 운영 규칙·예약 정책 설계

5. 구현 (Implementation)

5.1 MongoDB 및 Deadline Repository 설치·구성

5.2 Worker 환경 구성 및 그룹 세팅

5.3 PySide UI 구현 (PySide-based User Interface)

5.4 Google Sheets 예약 시스템 & GAS 구현

5.5 Maya/Blender Batch 렌더링 설정 구현 (Batch Rendering Configuration)

5.6 파이프라인 아키텍처 및 DCC 연동 구조

5.7 운영 자동화 및 보조 스크립트

6. 평가 및 결과 (Evaluation & Results)

6.1 실험 환경

6.2 성능 평가

6.3 운영 측면 평가

6.4 문제점 및 해결 요약

7. 결론 (Conclusion)

7.1 연구 요약

7.2 기여 정리

7.3 향후 연구 방향

8. 참고문헌

초록

본 연구는 홍익대학교 M211 실습실 환경을 대상으로, Autodesk Maya 및 Arnold 렌더러, Blender의 렌더링 작업을 효율적으로 수행하기 위한 Deadline 기반 렌더팜을 구축하고 자동화 시스템을 설계하는 것을 목표로 한다. 기존 실습실에서는 개별 PC에서 수동으로 렌더링을 수행하여 경로 설정 오류, 라이선스 충돌, 제출 기간의 병목 현상과 같은 문제가 지속적으로 발생하였다. 이러한 문제를 해결하기 위해 본 연구에서는 NAS 기반 중앙 저장소, Deadline Repository-Worker 구조, MongoDB를 활용한 렌더링 상태 관리, Google Apps Script 기반 예약 시스템, 그리고 PySide 기반 사용자 인터페이스(UI)를 통합한 렌더 자동화 워크플로우를 구현하였다. 구축 과정에서 NAS 권한 오류, Arnold 라이선스 인증 실패, MayaBatch UNC 경로 인식 문제, Firewall 포트 차단 등 다양한 기술적 이슈가 발생하였으며, 이를 해결하기 위한 설정 조정 및 네트워크 재구성을 수행하였다. 구축된 시스템은 다수의 학생이 동시에 렌더링 작업을 제출하는 상황에서도 안정적으로 동작함을 확인하였으며, 실습실 내 렌더링 효율성과 사용자 경험이 크게 향상되었다. 본 연구는 교육 환경에서의 렌더팜 설계 사례로서, 향후 GPU 노드 확장 및 자동 모니터링 시스템 도입 등 추가 연구의 기반을 제공한다.

홍익대학교 **M211** 실습실 환경을 위한 **Deadline** 기반 렌더팜 구축 및 렌더링 자동화
시스템 설계

Capstone Project Technical Paper

NAS·Deadline·MongoDB·PySide 기반 통합 렌더링 자동화 시스템 구현

1. 서론 (Introduction)

1.1 연구 배경

3D 애니메이션 및 영상 콘텐츠 제작 과정에서 렌더링은 전체 제작 파이프라인 중 가장 높은 연산 비용을 요구하는 단계이다. 교육 환경에서는 고성능 렌더 서버를 갖추기 어려워, 학생들이 개별 PC에서 수동 렌더링을 수행해야 하는 경우가 많다. 이러한 방식은 시간적 비효율뿐 아니라 반복적 오류 발생, 파일 경로 설정 문제, 네트워크 권한 문제 등 다양한 운영적 한계를 갖는다.

특히 홍익대학교 M211 실습실은 Maya, Arnold, Blender 기반의 렌더링 작업이 집중되는 시기마다 렌더 병목 현상, 파일 공유 불가 문제, 네트워크 경로 불일치, 라이선스 충돌 등이 반복적으로 발생해왔으며, 이는 학생들의 작업 품질과 프로젝트 마감 일정에 직접적인 영향을 미쳐왔다.

1.2 기존 실습실 렌더링 환경의 문제점

- NAS 파일 공유 불가 문제: 실습실 PC 간 파일 접근 권한이 통일되지 않아 경로 오류가 자주 발생함.
- 렌더 병목 현상: 마감 기간에 학생들이 몰리면 모든 장비의 렌더 큐가 막혀 평균 대기 시간이 크게 증가.
- Deep Freeze 초기화 정책으로 인해 모든 설정이 매번 초기화되어 렌더 환경을 구축하는 데 불필요한 시간이 반복됨.
- 라이선스 제약: Arnold는 Maya 2025 버전 라이선스만 허용되어, 맞지 않는 버전에서 렌더가 실패하는 문제가 빈번함.

- 파일 경로 불일치(UNC 미사용): 각자 다른 경로 구조에서 작업하며 MayaBatch가 UNC 경로를 인식하지 못해 렌더 실패가 잦았음.

1.3 연구 목적

본 연구의 목적은 다음과 같다:

1. Deadline 기반 렌더팜을 구축하여 학생들이 병렬 렌더링 환경을 사용할 수 있도록 한다.
2. PySide 기반 사용자 UI, MongoDB 기반 상태 관리, Google Sheets 기반 예약 시스템을 통합하여 실습실에 최적화된 자동화 파이프라인을 구축한다.
3. 반복적인 오류—파일 경로 문제, 권한 문제, 라이선스 충돌, Deep Freeze 초기화 문제—를 해결할 수 있는 일관된 파일 구조와 안정적인 네트워크 운영 환경을 마련한다.

1.4 시스템 구축 목표 및 의의

본 연구를 통해 구축된 시스템은 단순히 렌더링 속도를 향상하는 것을 넘어,
“교육 환경에 적합한 통합 렌더링 파이프라인 모델”을 제안하는 데 의의가 있다.

- 실습실 내 모든 작업을 예약 → 로그인 인증 → 렌더 제출 → 상태 확인 으로 구성된 단일 워크플로우로 통합.
- NAS 기반 경로 통일 정책(UNC 표준화) 적용으로 파일 접근 오류를 근본적으로 해소.

- MongoDB + Deadline 연동을 통해 Job 상태 변화 자동 추적, Google Sheets에 실시간 업데이트 가능.
 - 학습 환경에서 실제 스튜디오 파이프라인과 유사한 작업 경험을 제공하여 교육적 효과가 크다.
-

2. 관련 연구 및 기술 배경 (Related Work & Background)

2.1 Render Farm 개념 및 스튜디오 활용 사례

렌더팜(Render Farm)은 대규모 연산을 분산 처리하기 위해 여러 노드를 병렬로 운영하는 시스템이다. Pixar, Weta Digital, ILM 등 대형 VFX 스튜디오는 Deadline, Tractor, Qube! 등의 관리 툴을 기반으로 대규모 렌더팜을 운용하며, 프로젝트당 수천 대의 노드가 동시에 작업을 수행한다.

본 연구는 이러한 스튜디오 수준의 개념을 소규모 교육 환경에 최적화하여 적용한 사례이다.

2.2 Deadline 렌더 매니지먼트 시스템 구조

Deadline은 Thinkbox Software(현재 Amazon AWS 소속)의 분산 렌더링 관리 시스템으로, Repository-Database-Worker-Pulse 구조로 이루어진다.

주요 구성 요소는 다음과 같다:

- Repository: Job, Plugin, Script 파일을 관리하는 중앙 저장소.
- Database: Job 상태, Worker 정보 등을 관리하는 MongoDB 기반 데이터베이스.
- Worker: 실제 렌더링을 수행하는 노드.

- Pulse: 전체 Worker 상태를 모니터링하고 스케줄링을 개선하는 중앙 컨트롤러(선택적).

Deadline은 다양한 DCC(Maya, Blender, Houdini 등)를 지원하며 Python API를 제공하여 자동화 구현이 용이하다.

2.3 MayaBatch / Arnold / Blender 렌더 아키텍처

- MayaBatch: UI 없이 실행되는 Maya 렌더러로, Deadline에서 주로 사용하는 방식.
- Arnold: CPU 기반 고품질 렌더러이며, 라이선스 서버(RLM)를 통해 인증이 필요하다.
- Blender Cycles/Eevee: GPU·CPU 모두 지원하며 교육 환경에서 비교적 가볍게 렌더링 가능.

각 렌더러는 파일 경로가 절대 경로(UNC)로 통일되지 않으면 파일 로드 오류가 발생할 수 있다.

2.4 NAS·네트워크 구조·파일 경로 설계 개념

렌더팜에서는 모든 Worker가 동일한 파일 경로에 접근해야 일관된 렌더 결과를 얻을 수 있다.

본 연구는 NAS 기반 중앙 저장소를 구축하고, 다음과 같은 규칙을 적용하였다:

- UNC 경로 통일: [redacted] [redacted] [redacted], [redacted] [redacted]
- 폴더 구조 표준화: input / output / cache / project

- RAID 구성 구분: 임시 데이터는 RAID0, 결과물은 RAID1 기반 안정 저장소에 배치

이를 통해 기존 실습실에서 빈번했던 파일 접근 오류가 해결되었다.

2.5 PySide, MongoDB, Google Apps Script 기술 개요

- PySide6: UI를 제작하는 Python 프레임워크로, 사용자 렌더 제출 UI 개발에 사용됨.
- MongoDB: 로그인이력, 예약정보, 렌더 Job 상태를 저장하는 NoSQL DB.
- Google Apps Script(GAS): 예약 시스템을 자동화하며, Google Sheets와 MongoDB 간 데이터 동기화에 사용됨.

3. 시스템 요구사항 분석 (Requirements Analysis)

3.1 기능적 요구사항

- 학생 계정 기반 로그인 기능
- 예약 사용자만 렌더팜 사용 가능
- Maya/Blender 파일 업로드 및 렌더 설정 입력
- Deadline에 렌더 Job 자동 제출

- MongoDB에 렌더 상태 기록
- 상태 변화 시 Google Sheets로 자동 업데이트
- output 폴더 자동 생성 및 NAS 저장

3.2 비기능적 요구사항

- 예약자만 사용 가능해야 함 → 인증 필요
- NAS 접근이 항상 가능해야 함 → 경로 통일 필요
- 렌더 상태가 실시간으로 업데이트되어야 함
- DCC 버전 충돌 없이 실행되어야 함
- 실습실 특성상 접속 환경이 매번 초기화되어도 정상 작동해야 함

3.3 실습실 환경의 제약 조건

- Deep Freeze 정책: PC 재부팅 시 모든 설정 초기화 → 매번 새로 환경을 잡아야 하는 어려움
- NAS 권한 문제: 초기에는 파일 접근 불가로 렌더 실패

- Arnold 라이선스 제한: Maya 2025 버전에만 라이선스 제공
- 고정 IP 변경 이슈: 초기 개발 중 NAS·DB 주소가 자주 변경되며 시스템 불안정
- 네트워크 속도 제약: Worker 수가 증가할수록 병목 가능성 증가

3.4 기술 선택 이유

- Deadline: 교육 환경에서 가장 안정적이며 Python API가 풍부함
- MongoDB: UI와 Deadline 간 상태 동기화에 적합
- PySide: Windows 환경에서 간단히 UI 배포 가능
- Google Apps Script: 학생들이 접근하기 쉬운 예약 방식 + 자동화 가능

4. 시스템 설계 (System Design)

본 장에서는 요구사항 분석을 바탕으로 구축된 렌더링 자동화 시스템의 설계를 기술한다. 시스템 설계는 사용자 Workflow, Deadline 기반 아키텍처, NAS 폴더 구조, UI 구조, 데이터 흐름 설계의 다섯 영역으로 구성된다. 이 단계에서는 실제 구현 절차나 코드가 아닌, 전체 시스템이 어떻게 작동하도록 설계되었는지를 개념적으로 다룬다.

4.1 전체 Workflow 설계 (Overall Workflow Design)

본 연구의 렌더링 프로세스는 예약 → 인증 → 렌더 설정 → 상태 업데이트 → 결과 확인의 단계를 중심으로 설계되었다. 핵심 목적은 다수의 사용자가 동일한 실습실 환경에서

안정적으로 렌더 작업을 제출하고 결과를 확인할 수 있는 일관된 파이프라인을 제공하는 것이다.

Workflow 주요 구성 단계

1. **Google Sheets** 기반 예약 시스템

- 사용자 인증 및 예약 시간 검증을 담당
- 유효한 사용자만 렌더 실행 가능하도록 제한

2. **PySide UI** 로그인 및 렌더 설정 입력

- 예약 시간대에 211 실습실 PC에서 UI 로그인
- 장면 파일 경로, 출력 경로, 옵션(Command Flags) 입력 가능

3. **Deadline** 렌더 제출 및 **Worker** 병렬 처리

- 상태가 **queued**로 변경되는 순간 Deadline에 자동 Job 생성
- Worker는 NAS에서 파일을 읽어 프레임 단위 병렬 렌더 수행

4. 결과 전달 및 모니터링

- 사용자는 **Deadline Monitor**를 통해 각 Job의 세부 상태를 실시간으로 모니터링할 수 있다.

이 Workflow는 수동 렌더링과 달리,
사용자 행동을 통제하고 실수 가능성을 배제하도록 설계된 절차적 구조이다.

4.2 초기 아키텍처 검토 및 대안 평가 (Initial Architecture Exploration)

렌더팜을 구축하기 전에 여러 가능한 구조를 검토하였다. 특히 초기에는 Worker 환경을 **Docker** 컨테이너로 관리하고, 설치 및 업데이트를 **Ansible**로 자동화하는 방식을 고려하였다.

(1) Docker 기반 Worker 노드 구조 시도

처음에는 각 Worker PC마다

- MayaBatch / Blender CLI
- Deadline Worker
- Arnold Plugin
- OCIO Config

을 컨테이너 이미지로 통합하여 관리하려는 시도가 있었다.

이 접근은 다음 장점을 가졌다:

- 버전 관리가 쉬움
- 환경이 표준화됨

- 재설치/초기화가 1분 내로 가능
- 의존성 충돌 제거

그러나 실습실 환경에서는 다음 문제가 발생했다:

- Maya, Arnold 등 Autodesk 계열은 컨테이너 환경에서 정식 지원이 거의 없음
- GPU/라이선스/GUI 기반 인증 문제가 Docker와 충돌
- Deep Freeze 때문에 Docker Desktop 자체가 유지 불가
- 네트워크 드라이브(NAS) 마운트가 컨테이너에서 지속적으로 끊김

결론적으로 **DCC**툴 기반 렌더링은 **Docker**에 적합하지 않으며,
실제 테스트에서 MayaBatch가 컨테이너에서 정상 실행되지 않아 사용 불가 판정.

(2) Ansible 기반 Worker 자동 설치 시도

20대 Worker PC를 자동 세팅하려고 Ansible 플레이북도 작성했다:

시도한 작업:

- NAS net use 자동화
- Deadline Worker 설치

- 환경변수 등록
- Arnold License Env 설치 스크립트 배포
- 서비스 재시작 자동화

실제 문제점:

- Windows는 SSH 기반 Ansible 제어가 매우 불안정함
- 실습실 계정 정책으로 WinRM이 제한됨
- Deep Freeze 때문에 매 재부팅마다 초기화됨
- PC별 사용자 권한이 달라 Ansible이 100% 재현 실패함

4.3 Deadline 기반 아키텍처 설계 (Deadline Architecture Design)

Deadline은 Repository-Database-Client 구조를 기반으로 설계된 분산 렌더링 관리 시스템이다. 본 연구에서는 다음과 같은 형태의 아키텍처 배치를 채택하였다.

구조 구성요소

1. Deadline Repository

- Job Metadata, Plugin 설정, Worker 구성 정보가 저장됨
- NAS와 별도의 공유 폴더에 위치해 독립적 운영 가능

2. Deadline Database (MongoDB 기반)

- Repository와 함께 동작하며 Job 상태를 기록
- 본 연구에서는 UI용 MongoDB와 Deadline MongoDB를 분리하여 안정성 확보

3. Deadline Worker

- 20대의 실습실 PC를 4개 그룹으로 구성
- 각 Worker는 동일한 경로 구조의 NAS 파일을 읽어 병렬 렌더 수행
- Worker 그룹별로 라이선스 풀 또는 시간대 제약을 적용할 수 있도록 설계됨

4. Client (UI 제출 시스템)

- UI는 Job 데이터를 MongoDB에 기록하고, DB 상태 변경을 기준으로 Deadline에 Job이 자동 생성되도록 설계

설계 목적

- Worker 증감에 따라 수평 확장(Scalability)이 가능
- 사용자 개입 최소화
- 실습실 관리자가 특정 그룹만 사용하도록 배분 가능

4.4 NAS 경로 정책 및 폴더 구조 설계 (NAS Path & Folder Structure Design)

Worker 간 환경 차이로 인해 발생하는 경로 불일치 문제를 제거하기 위해 **UNC** 기반 절대 경로 표준화를 설계의 핵심 원칙으로 삼았다.

경로 정책 설계

- 모든 경로는 [redacted] ... 형식으로 통일
- 로컬 경로 사용 금지
- UI에서 경로 입력 시 자동으로 UNC 형태로 변환하도록 설계

폴더 구조 표준화 설계

```
/renderfarm
  /input
    /학번
      /날짜
        scene.mb / scene.blend ...
  /output
    /학번
      /날짜
        /frame_####.exr
```

설계 목적

- Worker 간 파일 접근 일관성 유지

- 사용자별/날짜별 기록 관리 용이
- 실습실 내 수백 Job이 누적되더라도 정리된 구조로 유지

4.5 UI 설계 (User Interface Design — Functional Overview)

PySide 기반 UI는 구현 세부 내용을 제외하고 다음과 같은 기능적 설계를 중심으로 구성되었다.

UI의 핵심 설계 요소

1. 로그인 모듈

- Google Sheets 예약 정보와 MongoDB 사용자 정보를 참조하여 유효 사용자만 로그인

2. 렌더 설정 입력 모듈

- Scene Path, Output Path, Frame Range, Resolution, Renderer 옵션 입력 가능하도록 설계
- 잘못된 입력을 방지하기 위해 경로 확인, 자동 폴더 생성 기능 포함

3. Job Preview 모듈

- 제출 전에 설정값을 시각적으로 검사하도록 설계

- 실수로 잘못된 위치에 렌더를 남기는 것을 방지

4. 제출 버튼 → MongoDB 업데이트

- Job 상태 초기값 = **queued**
- 상태 변경이 곧 Deadline의 Job 생성 트리거가 되도록 구성

UI는 사용자의 비전문성으로 인해 발생할 수 있는 오류를 설계 단계에서 차단하는 방향으로 구성되었다.

4.6 데이터 흐름 설계 (Data Flow Design)

본 시스템은 UI, MongoDB, Deadline, Google Apps Script(GAS) 간의 데이터 전달 흐름을 통해 자동화된 렌더파이프라인을 구성한다.

데이터 흐름 단계

1. UI → MongoDB

- 사용자가 입력한 렌더 요청이 document 형태로 저장
- 필드 예: { user, input_path, output_path, status, timestamp, options... }

2. MongoDB → Deadline

- MongoDB Change Stream을 활용하여 **status: queued** 변경을 트리거로 감지
- 감지 즉시 Python Listener가 Deadline 명령을 실행하여 Job 자동 생성

3. Deadline Worker → Output 저장

- Worker가 NAS의 input을 읽고, output을 동일 NAS 경로에 저장

4. Deadline → MongoDB (상태 업데이트)

- Worker 완료 시 UI DB에 **completed, error, running** 등의 상태 전달

5. MongoDB → Google Sheets (GAS)

- GAS가 DB 변경 사항을 주기적으로 읽어 예약표에 실시간 반영

설계 목표

- 모든 상태 변화가 자동으로 전달되도록 설계하여 수동 관리 부담 제거
- Google Sheets 환경에서 사용자가 웹 기반으로 결과 확인 가능
- UI와 Deadline 간 결합도를 낮춰 유지보수 용이

5. 구현 (Implementation)

본 장에서는 앞서 설계한 구조를 바탕으로 실제 렌더링 자동화 시스템을 구축한 과정을 기술한다. 구현은 Deadline 설치 및 설정, Worker 환경 구성, PySide UI 개발, MongoDB 기반 상태 관리, Google Apps Script 예약 시스템 구현, Maya/Blender Batch 렌더링 설정, 그리고 소프트웨어 파이프라인 아키텍처로 나누어 설명한다.

5.1 Deadline 설치 및 환경 구성 (Deadline Installation & Configuration)

5.1.1 MongoDB 및 Repository 설치·연결

본 시스템 구축의 출발점은 **MongoDB** 데이터베이스 서버 구성과 **Deadline Repository** 설치이다. Deadline 공식 문서에서는 “Quick Install” 형태의 간단한 설치 절차를 안내하지만, M211 실습실 환경(Deep Freeze, NAS, 제한된 권한, 방화벽 정책 등)에서는 다음과 같은 문제가 반복적으로 발생하였다.

1. MongoDB 버전 및 설치 이슈

Deadline이 지원하는 MongoDB 버전은 제한적이다. 최신 버전을 설치했을 때는

- Deadline Installer에서 MongoDB 인식 실패
- SSL handshake failed
- 서비스 등록 실패

와 같은 오류가 발생하였다. 이 때문에 여러 버전을 설치·제거하는 과정을 반복하며, Deadline 호환 표에 명시된 버전으로 다운그레이드하여 문제를 해결하였다.

2. 포트 충돌 및 방화벽 문제

MongoDB 기본 포트는 **27017**이다. 실습실 PC에는 기존에 사용 중인 서비스들이 존재해 포트 충돌 및 차단 문제가 발생했다.

- **netstat -ano** 명령으로 포트 점유 프로세스 확인
- Windows Defender 방화벽에서 27017 Inbound 규칙 허용
- MongoDB를 Windows 서비스로 재등록 및 기동 상태 확인

위 과정을 통해 Database 서버가 안정적으로 기동되도록 구성하였다.

3. bindIp 및 접근 권한 설정

초기 설정에서는 MongoDB가 [redacted] 만 허용하고 있어, 외부 Worker 및 UI 스크립트가 Database에 접근하지 못했다. **mongod.cfg**의 **bindIp** 옵션을 다음과 같이 수정하였다.

```
net:  
  port: 27017  
  bindIp: [redacted], [redacted]
```

실습실 내부망에서만 접근하도록 전제했기 때문에 인증 모드는 비활성화하고, Deadline용 DB 계정을 별도로 구성하여 Job 메타데이터를 저장하도록 했다.

4. Repository 설치 및 UNC 경로 문제

Deadline Repository를 네트워크 경로(예: `[redacted]`)에 직접 설치하려고 했을 때,

- “UNC paths are not valid repositories”
- Worker가 Repository를 찾지 못함
- Auxiliary 파일 업로드 실패

와 같은 문제가 발생했다. 따라서 먼저 로컬 디스크에 Repository를 설치한 뒤, 해당 폴더를 NAS/SMB 공유로 내보내는 방식으로 수정하였다. 이는 Deadline 공식 가이드에서 권장하는 방식이기도 하다.

5. 권한·Deep Freeze로 인한 설정 초기화 문제

Repository 경로는 NAS 공유와 Deep Freeze 정책이 동시에 영향을 미쳤다.

- Worker 계정이 Repository 쓰기 권한이 없어 설정 저장 실패
- Deep Freeze로 인해 재부팅 시 설정이 초기화

를 반복적으로 경험했다. 최종적으로는

- Repository 경로를 Deep Freeze 예외 목록에 추가

- 모든 Worker 계정에 Repository 공유 폴더에 대한 읽기/쓰기 권한 부여

를 통해 문제를 해결하였다.

6. Database-Repository 연결 안정화

설치 마법사 단계에서 다음과 같은 오류를 여러 차례 경험했다.

- “Could not connect to the database”
- “Database connection timed out”
- “Failed to verify MongoDB replica set status”

이 문제들은 대부분 방화벽 차단, 서비스 계정 설정, UNC 공유 누락에서 기인했다. 방화벽 예외 추가, MongoDB 서비스 계정 조정, Repository 공유 재설정 등을 반복적으로 수행한 뒤에야 Deadline Monitor에서 **Database Connection: OK, Repository Connection: OK** 상태를 확인할 수 있었다.

요약하면, 공식 문서상으로는 “Next를 누르면 설치가 끝나는” 단계이지만, 실습실 환경에서는 포트, 권한, Deep Freeze, NAS 정책이 얹혀 있어 설치 난이도가 매우 높았다.

5.2 Worker 설치 및 환경 구성 (Worker Node Setup & Configuration)

실습실 내 20대의 PC(1013호)를 Worker 노드로 구성하기 위해서는 단순히 Deadline Worker를 설치하는 것만으로는 충분하지 않았다. NAS 접근, 라이선스 인증, Arnold 환경 변수 설정, Deadline 인증서 등록 등의 추가 작업이 필요했다.

5.2.1 NAS 네트워크 연결 (net use 기반)

Worker는 렌더링 시 Scene, Texture, Cache를 모두 NAS에서 읽기 때문에, 부팅 후 자동으로 NAS에 연결되어야 한다.

- 관리자 권한 CMD 실행 후:

```
net use [redacted] /user:<계정> <비밀번호>  
/persistent:yes
```

- 연결 오류 시:

```
net use * /delete
```

후 다시 매핑한다.

이 작업을 통해 Worker가 **input/output** 디렉터리에 안정적으로 접근할 수 있게 하였다.

5.2.2 Deadline Worker 설치 및 초기 설정

각 PC에 Deadline Client를 설치하고 Worker를 활성화한 뒤, 다음과 같이 설정하였다.

- Repository 경로: [redacted]

- Database IP를 자동 검색이 아닌 직접 입력
- Worker 이름 규칙: [redacted] , [redacted] , ..., [redacted]

설치 이후 Deadline Monitor에서 각 Worker가 Online 상태이고, Repository/Database 연결이 정상인지 확인하였다.

5.2.3 Database 서버 연결 및 방화벽 설정

Worker가 Database에 접근하려면 다음 포트가 열려 있어야 한다.

- 27017 (MongoDB)
- 필요 시 Deadline Remote Connection 관련 포트

초기에는 방화벽으로 인해 “Failed to connect to database” 오류가 발생했다. 서버와 각 Worker에서 모두 포트 허용 규칙을 추가한 후, `deadlinecommand PingDatabase`로 연결 상태를 검증하였다.

5.2.4 Deadline 인증서(CRT/PFX) 등록

보안 통신을 위해 Worker는 Repository에 저장된 인증서를 로컬에 설치해야 한다.

- Repository에서 `Deadline10Client.pfx/.crt` 파일을
`C:\ProgramData\Thinkbox\Deadline10\certificate` 경로로 복사
- Deadline Launcher 재시작

인증서가 없을 경우 **SSL: unable to verify certificate** 오류가 발생하며 Repository 접근이 불가하다.

5.2.5 Arnold 라이선스 환경 변수 설정

Arnold는 RLM 기반 네트워크 라이선스를 사용하므로, Worker마다 아래 환경 변수가 필요하다.

- **ADSKFLEX_LICENSE_FILE=27000@<license-server-ip>**
- **solidangle_LICENSE=5053@<license-server-ip>**
- **ARNOLD_PLUGIN_PATH, MTOA_EXTENSIONS_PATH, MTOA_TEMPLATES_PATH**
등

이 값들을 일괄로 설정하기 위해 **set_arnold_license_env.bat** 스크립트를 제작하여 관리자 권한으로 실행했다. 설정이 올바르게 없을 경우 Worker에서는 “Arnold License not found” 오류가 발생하며 MayaBatch 렌더가 실패한다.

서버 측 포트는 PowerShell에서 다음 명령으로 확인하였다.

```
Test-NetConnection -ComputerName [redacted] -Port 5053
```

5.2.6 RLM 네트워크 라이선스 서버 상태 점검

서버([redacted])에서는 **rlm.foundry.exe** 프로세스와 5053/5054 포트가 정상적으로 열려 있는지 확인하였다.

- **netstat -an | find "5053"**

- 방화벽 예외 설정
- 문제 발생 시 RLM 재시작 및 Worker 측 환경변수 재검증

5.2.7 Worker 그룹 구성

작업 분배 효율성을 위해 Worker를 용도별 그룹으로 나누었다.

그룹	PC 수	용도
A	5대	Maya CPU 렌더
B	5대	Blender CPU 렌더
C	5대	혼합(Overflow)
D	5대	테스트/유지보수

Deadline Pool/Group 설정과 연동하여, 특정 시간대에는 특정 그룹만 사용하도록 제어하였다.

5.3 PySide UI 구현 (PySide-based User Interface)

PySide UI는 사용자 입력을 최소화하고, 렌더 제출 절차를 표준화하기 위한 핵심 요소이다. 본 시스템의 제출 UI는 “UI 실행 → 로그인 → 곧바로 Submit 창”이라는 단순한 흐름으로 설계되었다.

5.3.1 UI 동작 흐름

1. 사용자가 UI를 실행하면 로그인 화면이 표시된다.

2. 학번·비밀번호·예약 시간 검증이 완료되면, 추가 메뉴 없이 즉시 “Submit to Deadline” 창이 열린다.
3. 사용자는 씬 파일, 출력 경로, 렌더 옵션 등 최소한의 항목만 입력하고 바로 Job을 제출할 수 있다.

이는 기존 Maya/Blender 내부의 Deadline Submitter와 유사한 경험을 제공하면서도, 실습실 환경에서 불필요한 옵션은 숨겨 사용성을 높이기 위한 설계이다.

5.3.2 필수 입력 항목

Submit 창에서는 다음 항목을 입력하거나 선택한다.

- Scene File 경로
- Output 경로 (필요 시 자동 생성)
- Frame Range
- Resolution
- Camera
- Renderer 옵션 (Arnold / Maya Software / Blender Cycles)
- Pool (Worker 그룹 선택)
- Concurrent Tasks

- Task Timeout (기본값: 5000초)

5.3.3 사용자 오류 방지 기능

실습실에서 반복적으로 발생하던 실수들을 분석하여, UI 단계에서 최대한 차단하도록 구현하였다.

1. 프로젝트 폴더 전체 제출 방지

- 허용 확장자를 **.mb**, **.ma**, **.blend**로 제한
- 폴더를 Scene File로 선택할 경우 즉시 경고창 표시

2. NAS 외 경로 사용 금지

- Scene / Output 경로가 반드시

[redacted]

... 내부인지 검사

- **C:**, **D:** 등의 로컬 경로는 자동으로 거부
- 경로 내 공백(), 불필요한 .가 포함된 경우 제출 차단
 - 예: **scenes.mb\...**, **s03 c220\Lighting** 은 오류 가능성이 높으므로 방지

3. 학번별 **input** 디렉터리 자동 매핑

학생별로 지정된 **input** 경로를 내부에서 매핑하여, 사용자가 경로를 직접 타이핑하지 않도록 하였다.

학번	자동 매핑된 input 경로
[redacted]	[redacted] arm\[redacted] \input
[redacted]	[redacted] arm\[redacted] \input
...	...

4.

Worker 상태 안내

렌더 성공을 위해서는 M211 실습실의 Worker PC가 모두 켜져 있어야 한다. Submit UI에는 다음 안내 문구를 포함하였다.

- “잡 제출 전 M211실의 모든 Worker PC가 켜져 있는지 Deadline Monitor에서 확인해 주세요.”

5.3.4 제출 후 검증 절차(Deadline Monitor 연동)

Submit 이후 운영 절차는 다음과 같이 표준화하였다.

1. Deadline Monitor에서 본인이 제출한 Job Name 선택
2. Task 탭에서 각 Task의 실행 시간 확인

3. Task Time이 비정상적으로 짧을 경우(예: 1분 30초 미만) 로그를 열어 오류 여부 확인
4. 로그를 Notion 템플릿에 붙여넣고 담당자에게 공유

UI에서는 이 절차를 안내 문구로 표시하여, 사용자가 매번 동일한 방식으로 Job을 검증하도록 유도하였다.

5.4 Google Sheets 기반 예약 시스템 구현 (Google Apps Script Integration)

본 시스템의 렌더팜은 다수의 사용자가 동시에 공유하는 인프라이기 때문에, 예약 관리와 사용 규칙의 자동화가 필수적이다. 이를 위해 Google Sheets와 Google Apps Script(GAS)를 사용하여 렌더 예약, 사용자 인증, 사용 규칙 적용, 상태 모니터링 기능을 제공하는 관리 시스템을 구현하였다.

Google Sheets는 접근성이 높고, 실습실 사용자 전원이 이미 익숙한 UI 환경을 갖고 있다는 점에서 적합한 선택이었다. 또한 GAS는 스프레드시트 기반의 서버리스 자동화 로직을 작성할 수 있어 렌더팜 운영 정책을 쉽게 시스템화할 수 있었다.

5.4.1 렌더 타임블록 구조 및 정책 반영

렌더팜의 자원은 다음과 같이 운영된다.

- 1일 4타임(19~23, 23~03, 03~07, 07~11)
- 타임당 4개 PC 그룹 운영 → 16블럭/일(평일)

- 주말은 **24시간 운영** → **24블럭/일**

총 가용량은 다음과 같다.

구분	블럭 수
----	------

평일(5일)	80블럭
--------	------

주말(2일)	48블럭
--------	------

총합	128블럭/주
----	----------------

이 구조는 Google Sheets 예약 UI에 그대로 반영되며, 스크립트는 다음 요구조건을 자동으로 검증한다:

- 각 팀이 확보할 수 있는 블럭 수의 공정한 분배
- 특정 팀의 과도한 선점 방지
- 베타 테스트 기간 동안 팀당 3블럭 제한 적용
- 주 단위 사용량 집계 및 로그 기록

5.4.2 예약 인증 로직 (GAS 기반)

GAS는 예약 시스템에서 다음 기능을 수행한다.

① 사용자 인증

- 제출된 학번(**ID**)을 스프레드시트 등록 정보와 대조
- 비밀번호(암호화 저장)를 검증
- 유효 사용자만 예약 가능하도록 제한

② 중복 예약 방지

- 동일 사용자가 동일 타임블럭을 두 번 예약하는 것을 차단
- 타인의 블럭을 수정하거나 덮어쓰는 행위를 감지하여 즉시 로그 기록
- 반복될 경우 자동 페널티 부여

③ 예약 가능 시간대 제한

- 이미 지난 시간은 예약 불가
- 베타 테스트 기간 또는 특정 이벤트 기간에는 추가 제한 부여
- 팀당 예약 가능 블럭 수를 초과하면 자동 거부

5.4.3 페널티 시스템 자동화

Google Sheets와 GAS는 단순 예약 시스템을 넘어

실습실 운영 규칙을 자동으로 적용하는 감시·관리 시스템 역할도 수행한다.

GAS는 다음 항목을 자동 검사하고 로그화한다:

위반 유형	자동 감지 여부
타 학번으로 예약	O
타인의 예약 수정	O
동일 팀 과다 예약	O
예약-취소 반복 스캠	O
운영 시간 위반	O

검출 시 로그 시트에 기록되며 패널티가 부여된다:

누적 횟수	패널티
1회	경고
2회	해당 주 라이선스 -2블럭
3회	3일 사용 정지
4회 이상	지정 기간 팀 사용권 박탈

이는 실제 운영 경험을 기반으로 구축한 공정성 유지 알고리즘이다.

5.5 Maya/Blender Batch 렌더링 설정 구현 (Batch Rendering Configuration)

본 연구에서는 Maya와 Blender 기반의 Batch Rendering 프로세스를 Deadline과 연동하여 자동화하였다. 각 DCC(Digital Content Creation) 도구는 렌더 팜 워커에서 CLI 기반으로 실행되며, NAS 및 환경 변수를 기준으로 일관된 렌더링 환경을 유지하도록 구성하였다.

5.5.1 MayaBatch 설정 (Maya + Arnold Integration)

① Arnold NLM(Network License Manager) 라이선스 등록 과정

실습실 환경에서는 Arnold를 포함한 Autodesk 제품군의 라이선스를 네트워크 라이선스(NLM) 방식으로 운영하였다.

라이선스 구성 절차는 다음과 같다:

1. Autodesk License File 생성 및 병합

Autodesk 공식 툴을 사용하여 여러 제품의 라이선스를 하나의 파일로 병합하였다.
(참고: *Combine License Files*¹⁾)

2. 서버 측 NLM 활성화

- License Server Manager(Adskflex)를 실행하고
- **adskflex.exe**를 통해 Arnold 포함 Maya 라이선스를 서비스로 등록
- 포트(**27000@<server-ip>** 등) 확인 및 활성화

클라이언트 PC 환경 변수 설정

각 워커 PC는 아래 두 환경 변수를 가지도록 구성하였다:

```
ADSKFLEX_LICENSE_FILE=27000@<license-server-ip>  
solidangle_LICENSE=5053@<license-server-ip>
```

3. **Arnold**는 별도의 포트(**5053**)를 사용하므로 이 설정이 중요하다.

```
Test-NetConnection -ComputerName [redacted] -Port 5053
```

위 설정으로 모든 워커에서 **Arnold** 렌더러가 정상적으로 **Batch**에서 라이선스를 획득하며 동작하였다.

② Deadline과 Maya의 Submitter 연동

Deadline Repository를 설정한 후,

Deadline Plugin Installer를 실행하면 Maya/Blender 등의 DCC 툴이 자동으로 연동된다.

- Maya 메뉴에 “**Submit to Deadline**” 버튼이 자동 생성됨
 - 별도 스크립트 설치 없이 모든 워커에 기본 구성 완료됨
-

5.5.2 Blender Batch 설정 (Blender + Deadline Integration)

Blender는 Deadline에서 자동 설치되는 플러그인만으로 즉시 호환되었으며 추가 설정이 거의 필요하지 않았다.

구성 방식은 다음과 같다:

Cycles 렌더러 **CPU** 모드 기반 **Batch** 렌더링

```
blender -b <scene> -E CYCLES -o <output> -f <frame>
```

1. UI에서 받은 **Output Path**를 **CLI** 옵션으로 전달

Blender는 경로 파싱이 Maya보다 안정적이라 추가 패치 필요 없음.

2. **Deadline** 플러그인 설치 시 자동 연동됨

Maya와 달리 수동으로 스크립트를 추가할 필요가 거의 없었다.

5.6 파이프라인 아키텍처 및 **DCC** 연동 구조

본 연구에서 구축한 렌더팜 시스템은 단일 스크립트가 모든 기능을 처리하는 방식이 아니라, 계층형 아키텍처를 기반으로 설계되었다. 전체 구조는 크게 ① 사용자 인터페이스 계층 ② **DCC** 어댑터 계층 ③ 렌더 제출 및 프리플라이트(**core**) 계층 ④ 인증·예약 및 외부 서비스 연동 계층으로 나뉜다. 이러한 분리는 Maya와 Blender라는 서로 다른 **DCC** 툴을 하나의 파이프라인 아래에서 통합하기 위함이며, 동시에 유지보수성과 확장성을 확보하기 위한 설계 선택이다.

5.6.1 계층형 아키텍처 개요

가장 상위에는 사용자가 직접 마주하는 **UI** 계층이 위치한다. 이 계층은 PySide 기반으로 구현된 로그인 창과 제출 창, 그리고 Maya 메뉴 상단에 추가되는 “**PRFS**” 메뉴로 구성된다. 사용자는 **PRFS** 메뉴에서 “**Submit to Deadline**”을 선택하면 로그인 UI가 열리고, 로그인 성공 후 제출 UI를 통해 썬 파일, 예약 시간, 제출 정보 등을 확인한 뒤 렌더를 요청한다. 이 계층은 가능한 한 단순한 이벤트 처리와 화면 표시만 담당하도록 설계되었으며, 실제 비즈니스 로직은 모두 하위 계층으로 위임된다.

UI 계층 아래에는 **DCC** 어댑터 계층과 렌더 제출(**core**) 계층이 존재한다. 어댑터 계층은 Maya와 Blender처럼 서로 다른 **DCC** 환경에서 썬 정보와 렌더 설정을 추출하는 역할을 수행하고, **core** 계층은 이 정보를 바탕으로 공통 데이터 구조를 구성하고 Deadline에 제출하는 책임을 진다. 그 아래에는 MongoDB와 Google Sheets, Deadline Repository 및

Worker로 구성된 인프라 계층이 존재하여, 인증 정보 관리, 예약 정보 동기화, 렌더 Job 저장 및 실행을 담당한다.

이러한 계층 구조를 통해 UI는 DCC 종류나 Deadline 세부 설정을 몰라도 되고, core는 DCC별 API에 직접 의존하지 않으며, 인프라 계층은 상위 로직과 분리된 상태로 독립적으로 관리할 수 있다.

5.6.2 DCC 어댑터 기반 구조

Maya와 Blender는 내부 데이터 구조와 API가 크게 다르기 때문에, 두 환경을 하나의 렌더팜 파이프라인으로 통합하기 위해 어댑터 패턴을 적용하였다. `gpclean_submit` 패키지의 `dcc` 모듈에는 추상 클래스 `DCCAdapter`와 이를 상속하는 `MayaAdapter`, `BlenderAdapter`가 정의되어 있다.

`DCCAdapter`는 `gather_scene_info()`, `gather_render_settings()`, `validate()` 세 가지 메서드를 추상 메서드로 선언한다. 각 DCC 어댑터는 이 인터페이스를 구현하여 자신이 담당하는 DCC 환경에서 씬 파일 경로, 프로젝트 루트, 렌더러 이름, 프레임 범위, 출력 경로 등 필수 정보를 수집한다. 예를 들어 `MayaAdapter`는 `cmds.file`과 `cmds.workspace`를 사용하여 현재 열려 있는 씬의 경로와 프로젝트 루트를 얻고, 렌더러와 카메라, 렌더 레이어 정보를 Maya의 렌더 설정 노드에서 직접 읽어온다. 반면 `BlenderAdapter`는 `bpy.data.filepath`와 `bpy.context.scene.render`를 통해 동일한 정보를 Blender 방식으로 추출한다.

추출된 정보는 `SceneInfo`, `RenderSettings`와 같은 데이터 클래스로 정규화된다. 이로 인해 상위 계층(core)은 Maya인지 Blender인지에 관심을 가지지 않고, 오직 공통 구조를 통해 데이터를 처리할 수 있다. 또한 `validate()` 메서드는 Arnold 플러그인 로드 여부, 프레임 범위의 유효성 등 DCC별로 필요한 사전 검증을 수행하여, 프리플라이트 단계에서 즉시 오류를 발견하고 사용자가 수정할 수 있도록 한다. 이러한 어댑터 기반

설계는 향후 Houdini, Nuke 등 다른 DCC 툴을 추가하더라도 동일한 인터페이스를 구현하는 것만으로 파이프라인에 편입할 수 있다는 확장성 측면의 장점을 제공한다.

5.6.3 인증 및 예약 정보 연동 구조

실습실 환경에서는 동시에 많은 학생이 렌더팜을 공유하기 때문에, 사용자 인증과 예약 정보 관리가 필수적이다. 본 연구에서는 MongoDB를 중심으로 인증 정보와 예약 정보를 관리하고, Google Sheets와 연동하여 사용자가 웹 환경에서도 예약 현황을 확인할 수 있도록 구성하였다.

`backend/authDB` 모듈은 MongoDB 연결과 컬렉션 객체 생성을 담당한다.

`auth_collection`은 학생별 로그인을 위한 학번 및 비밀번호 해시 값을 저장하며, 평문 비밀번호 대신 SHA-256 해시를 사용함으로써 기본적인 보안을 확보하였다. 로그인 UI에서 입력된 학번과 비밀번호는 컨트롤러를 통해 해시 검증을 거친 후, 일치하는 경우에만 렌더 제출 UI로 진입할 수 있다. `reservation_collection`은 Google Sheets에서 관리하던 예약 정보를 주기적으로 읽어와 동기화한 데이터베이스로, 로그인한 사용자가 어느 시간대에 렌더팜 사용 권한을 가지고 있는지 확인하는 기준이 된다.

Google Sheets와 MongoDB 간의 동기화는 별도의 스크립트를 통해 수행된다. 스크립트는 Google Sheets API를 사용하여 예약 스프레드시트에서 각 학번별 예약 블록을 읽은 뒤, 이를 MongoDB의 `reservation_collection`에 저장한다. 제출 UI는 이 정보를 기반으로 현재 로그인한 학생이 예약한 타임 블록 목록을 보여주며, 사용자가 예약하지 않은 시간대에는 Submit 버튼을 비활성화하거나 경고 메시지를 표시할 수 있다. 이 구조는 단순히 렌더를 제출하는 도구를 넘어, 렌더팜 사용 정책과 공정성을 시스템 수준에서 강제하는 기반을 제공한다.

5.6.4 설계 의의

요약하면, 본 시스템은 UI와 비즈니스 로직, DCC별 구현, 인프라 계층을 명확히 분리하고, DCC 어댑터와 공통 데이터 구조를 통해 Maya와 Blender를 통합하는 파이프라인 지향 설계를 채택하였다. 또한 MongoDB와 Google Sheets를 활용한 인증·예약 연동을 통해, 실습실 렌더팜 운영 정책을 코드 수준에서 구현하였다. 이러한 아키텍처는 단기적인 졸업 작품용 도구를 넘어, 향후 다른 DCC 툴이나 추가적인 검증 로직을 쉽게 편입할 수 있는 기반을 제공한다는 점에서 교육용 렌더팜 시스템으로서의 의의를 가진다.

5.7 운영 자동화 및 보조 스크립트 (Operational Automation & Helper Scripts)

본 연구에서는 렌더팜을 안정적으로 운영하기 위해, 단순한 렌더 제출 기능 외에도 환경 초기화, 라이선스 설정, 로그 수집을 자동화하는 여러 보조 스크립트를 제작하였다. 이들 스크립트는 실습실과 같이 사용자 교체가 잦고, 설정이 쉽게 깨지는 환경에서 운영 비용을 줄이고 장애 대응 속도를 높이는 데 중요한 역할을 한다.

5.7.1 OCIO 초기화 스크립트 (OCIO Reset Script)

Maya 2025 환경에서는 OpenColorIO(OCIO) 설정이 손상되거나 잘못된 경로를 가리키는 경우, 다음과 같은 오류가 반복적으로 발생하였다.

- `could not read '...config.ocio' OCIO profile.`
- 썬 로딩 또는 렌더 시 경고/에러 출력
- Deadline MayaBatch 렌더 실패

실습실 PC는 Deep Freeze 정책으로 인해 재부팅 시 설정이 초기화되지만, 사용자별 환경 설정이 꼬이거나 잘못된 경로를 참조하는 문제는 계속 반복될 수 있었다. 이를 해결하기 위해 다음과 같은 **OCIO** 초기화 스크립트를 제작하였다.

- Maya가 참조해야 하는 정상적인 **config.ocio** 경로를 기준값으로 정의
- 잘못된 경로 또는 손상된 설정이 감지될 경우:
 - 환경 변수(**OCIO**)를 올바른 경로로 재설정
 - Maya 환경 설정 파일에서 잘못된 OCIO 설정 항목 제거 또는 덮어쓰기
- Deadline Batch 렌더링 시에도 동일한 경로를 사용하도록, Worker 환경에서 공통 설정을 유지

이 스크립트는 “렌더가 갑자기 이상한 색으로 나오거나, OCIO 관련 에러가 로그에 반복되는 경우”에 조교·관리자가 한 번 실행하여 즉시 상태를 초기화할 수 있는 복구 도구로 동작한다. 그 결과, 색공간 이슈로 인한 렌더 실패 비율을 크게 줄일 수 있었다.

5.7.2 Arnold 라이선스 환경변수 설정 스크립트 (**set_arnold_license_env.bat**)

앞서 설명한 것처럼 Arnold 렌더러는 네트워크 라이선스 서버(RLM)를 통해 라이선스를 인증받는다. 이때 각 Worker PC에는 다음과 같은 환경 변수가 올바르게 설정되어 있어야 한다.

- **ADSKFLEX_LICENSE_FILE**
- **solidangle_LICENSE**
- **ARNOLD_PLUGIN_PATH**

- `MTOA_EXTENSIONS_PATH`
- `MTOA_TEMPLATES_PATH`

초기에는 이 값을 수동으로 입력하면서 오타, 경로 실수, 사용자별 설정 누락 등 문제가 잦았다. 실습실과 같이 PC 수가 많은 환경에서는 수동 작업이 사실상 불가능했기 때문에, 이를 자동화하기 위한 `set_arnold_license_env.bat` 스크립트를 제작하였다.

스크립트의 주요 동작은 다음과 같다.

- 관리자 권한으로 실행 시, 시스템 환경 변수에 위 항목들을 일괄 설정
- 라이선스 서버 IP와 포트(예: `5053@redacted`)를 고정값으로 등록
- Arnold 플러그인 및 템플릿 경로를 Maya 설치 경로와 연동하여 세팅
- 설정 후, 간단한 테스트 렌더 또는 `kick` 명령 등을 통해 라이선스 인식 여부를 확인할 수 있도록 안내 메시지 출력

이 스크립트를 도입한 이후에는 **Worker** 한 대를 새로 세팅하는 작업이 “스크립트 실행 한번” 수준으로 단순화되었으며, 라이선스 미설정으로 인한 렌더 실패 케이스도 크게 감소하였다.

5.7.3 로그 수집 및 Notion 업로드 프로세스

실습실 렌더팜 운영에서 가장 중요한 것 중 하나는, 에러가 발생했을 때 원인을 빠르게 파악하고 재발을 막는 것이다. 이를 위해 Deadline의 Job/Task 로그를 수동으로 확인하는

단계에 더해, 로그를 모아 구조적으로 기록하는 **Notion** 기반 에러 관리 프로세스를 설계하였다.

운영 흐름은 다음과 같다.

1. 사용자가 Deadline Monitor에서 자신의 Job을 선택한다.
2. Task 탭에서 특정 Task에 이상 징후가 있을 경우(예: Task Time이 비정상적으로 짧거나, Status가 Failed인 경우), 해당 Task를 더블 클릭하여 로그를 연다.
3. 로그 전체를 드래그하여 복사한 뒤, 미리 준비해둔 Notion 템플릿 페이지(예: 학번_날짜_샷번호)에 붙여넣는다.
4. Notion 템플릿에는 다음 필드가 포함된다.
 - 학번 / 팀명
 - 렌더링 날짜 및 시간대
 - 사용하는 DCC 와 렌더러(Maya+Arnold / Blender+Cycles 등)
 - 사용한 Pool / Worker 그룹
 - 에러 로그 전문
 - 담당 조교 의견 / 해결 방법 메모

운영을 돕기 위해, 다음과 같은 보조 요소도 함께 설계하였다.

- Notion에서 새 페이지를 생성할 때, 학번_날짜 형식으로 제목을 자동 추천하여 정렬과 검색을 쉽게 함
- 자주 발생하는 에러 유형(예: NAS 경로 미연결, 라이선스 실패, OCIO 설정 오류)을 태그로 등록하여, 비슷한 오류를 빠르게 검색 가능
- 조교가 해결 방법을 기록해 두면, 이후 같은 에러가 발생했을 때 학생이 기존 해결 사례를 보고 자가 해결할 수 있음

이 프로세스는 엄밀한 의미의 “스크립트”라기보다는, **Deadline Monitor + Notion** 템플릿 + 운영 규칙이 결합된 워크플로우이지만, 렌더팜 운영을 자동화·표준화한 중요한 구성 요소이다. 단순히 에러를 “그때그때 보는 것”에서 나아가, 시간이 지날수록 축적되는 에러·해결 사례 데이터베이스로 발전할 수 있다는 점에서 교육용 렌더팜 운영에 큰 의의를 가진다.

6. 평가 및 결과 (Evaluation & Results)

본 장에서는 구축한 Deadline 기반 렌더팜 시스템을 실제 실습실 환경에서 운영하며 얻은 성능, 안정성, 사용자 측면의 운영 결과를 정리한다. 또한 구축 과정에서 발생한 주요 문제점과 이를 해결한 과정도 함께 기술함으로써, 제안 시스템의 실효성과 개선 가능성을 검증한다.

6.1 실험 환경 (Experimental Environment)

본 시스템은 홍익대학교 M211 컴퓨터 실습실을 기반으로 구축되었으며, 다음과 같은 환경에서 평가되었다.

하드웨어 환경

- Worker PC: 총 20대
 - Cpu 12 core processors 20
 - Intel(R) UHD Graphics 770 NVIDIA GeForce RTX 3080
- 서버 PC (M211-00):
 - Deadline Repository + MongoDB Database + RLM License Server 통합 운영
 - NAS와 동일 네트워크 구성

소프트웨어 환경

- Deadline 10.4
- MongoDB 5.x
- Maya 2023 / Maya 2024 / Maya 2025 / Blender 4.x
- Google Sheets + Google Apps Script

- PySide6 기반 제출 UI
- Windows 10 + Deep Freeze 정책 적용

테스트 데이터

- 졸업작품용 Maya 씬 (조명·FX·Alembic 포함)
- Blender Cycles 씬 (볼륨/머티리얼 포함)
- 평균 프레임 시간: 3분 27

6.2 성능 평가 (Performance Evaluation)

본 성능 평가는 세 가지 기준으로 이루어졌다.

1. 프레임 처리 속도(실제 **Deadline** 로그 기반)
2. **Worker** 그룹 구성 효과
3. 오류율 개선 효과(사전·사후 비교)

평가는 실제 실습실 환경의 졸업작품 씬 6종(Maya 4개, Blender 2개)을 대상으로 진행되었으며,
각 씬은 평균 120~240프레임 범위에서 테스트됐다.

(1) 렌더링 처리 속도 평가 (Frame Throughput Based on Deadline Task Logs)

단일 PC에서 렌더링할 때

학생들이 개인 PC에서 수동으로 렌더링할 때는 다음과 같은 평균 프레임 시간이 기록되었다:

DCC	평균 프레임 시간(개인 PC)
Maya + Arnold (조명 썬)	2분 10초 ~ 3분 40초
Blender Cycles (Simulation 포함)	1분 40초 ~ 2분 20초

→ 평균적으로 1프레임 약 120~220초 소요됨.

렌더팜(20 Worker)에서 병렬 렌더링할 때

Deadline Monitor의 Task 패널 기준 측정 값:

DCC	Task Execution Time 범위(20 Worker)
Maya + Arnold	23초 ~ 52초/프레임
Blender Cycles	18초 ~ 44초/프레임

그래서 실제 향상 폭은 “프레임당 2~3배 빠름”이 아니라,

→ 20대가 동시에 나눠 처리하기 때문에 총 완성 시간 기준으로는 20배 가까이 단축

실제 예시: S003_c120 썬(240프레임) 기준

방식	총 소요 시간
개인 PC 단독 렌더	약 9시간 10분

렌더팜 20 Worker 병렬 약 **26~32**분

즉, 총 프로젝트 단위에서는 약 **17~20**배의 시간 단축 효과가 있었다. 위 측정값은 모두 Deadline Monitor의 Task Log에서 실제 완료 시간을 수집하여 평균을 낸 값이다.

6.3 운영 측면 평가 (Operational Evaluation)

(1) 사용자 경험 개선

- 기존에는 Maya 내부에서 직접 경로를 지정해 렌더 제출 → 오류 가능성 높음
- PySide UI의 강제적 경로 검증 + 학번 자동 매핑 으로 사용자 실수가 거의 사라짐
- 로그인 후 즉시 ‘Submit to Deadline’ 창이 열리는 구조는 초보 사용자에게 매우 효율적임

(2) 예약 시스템 효과

- Google Sheets 기반 예약 시스템은 실습실 자원 배분의 공정성을 높였고
중복 예약·잘못된 예약·타 학번 예약 등의 문제를 자동 감지하여 운영 부담을 크게 줄였다.

(3) 조교 운영 효율 개선

- Notion 기반 로그 기록 시스템 덕분에
 - 오류 패턴 추적
 - 개선 기록 누적

- 사용자 교육 자료 구축

이 가능해져, 매 학기 유지보수 난이도가 크게 감소할 것으로 예상된다.

6.4 문제점 및 해결 요약 (Issues & Resolution Summary)

아래는 구축 과정에서 특히 난이도가 높았던 문제와 해결 방법이다.

문제	원인	해결
MongoDB 연결 실패	포트 차단, bindIp 설정 오류	방화벽 허용, config 수정
Deadline Repository UNC 인식 실패	네트워크 공유 권한 부족	로컬 설치 후 공유 방식으로 변경
Worker가 라이선스 인식 실패	Arnold 포트(5053) 미허용	환경 변수 자동 스크립트 제작
OCIO 오류로 인한 MayaBatch 실패	잘못된 config.ocio 참조	OCIO reset 스크립트 제작
학생들의 빈번한 제출 오류	경로·확장자 실수	PySide 제출 UI에서 강제 검증 도입
Worker Offline로 인한 대량 실패	PC 꺼짐 상태 미검증	운영 매뉴얼 개선 및 안내 강화

7. 결론 (Conclusion)

본 연구는 교육용 실습실 환경에서 안정적으로 동작하는 Deadline 기반 렌더팜 시스템을 구축하고, 이를 운영하기 위한 UI·DB·자동화 스크립트를 통합적으로 설계·구현한 사례 연구이다.

7.1 연구 요약

본 프로젝트는 다음 세 가지 구성 요소를 중심으로 설계되었다.

1. 인프라 구축:

- MongoDB + Deadline Repository
- 20대 Worker 구성
- Arnold NLM, NAS 기반 파일 구조 정립

2. 사용자 인터페이스 및 제어:

- PySide UI 기반 제출 시스템
- 자동 경로 검증, 학번 기반 input 매핑
- Maya/Blender 어댑터 기반 통합 렌더 파이프라인

3. 운영 자동화 및 관리 시스템:

- Google Sheets 예약 시스템
- GAS 기반 예약 검증 및 페널티 시스템
- Notion 기반 오류 로그 DB화

이를 통해 교육용 환경에서도 실제 스튜디오에 가까운 렌더링 파이프라인 구축이 가능함을 보였다.

7.2 기여 정리 (Contributions)

본 연구의 기여는 다음과 같다. 초기에는 Docker 및 Ansible 기반 자동화를 검토했으나, 라이선스·Deep Freeze·WinRM 제약 등으로 실습 환경에 적합하지 않다고 판단하고, 네이티브 Worker 기반 아키텍처를 채택하였다. 이는 향후 다른 환경에 이식할 때 고려해야 할 설계 trade-off 사례로 의미가 있다.

기술적 기여

- Maya/Blender를 하나의 파이프라인으로 묶는 **DCC Adapter** 패턴 적용
- PySide 기반의 제출 **UI + Deadline** 통합 자동화
- 환경 변수·라이선스·OCIO 문제를 자동 복구하는 스크립트 개발
- NAS-Repository-Worker 구조의 정교한 재설계 및 안정화

운영적 기여

- Google Sheets + GAS 기반 공정한 예약 시스템 구축
- 오류 로그의 DB화(Notion)로 운영 지식이 지속적으로 축적
- 학생 사용자들의 실수를 UI 수준에서 제거
- 조교 및 관리자 운영 부담 대폭 감소

교육적 기여

- 실제 스튜디오 워크플로우에 근접한 파이프라인 체험 제공
- 렌더링 파이프라인과 시스템 운영의 원리를 학생이 직접 이해할 수 있도록 함

7.3 향후 연구 방향 (Future Work)

본 시스템은 이미 안정적으로 운영되고 있으나, 다음과 같은 확장이 가능하다.

1. GPU 기반 워커 추가 및 **Blender Cycles X** 가속 최적화
2. **Houdini / Nuke**와 같은 추가 **DCC** 툴 지원(**DCC Adapter** 확장)
3. 웹 기반 제출 UI(**Web Frontend + FastAPI**)로 확장
4. 자동 오류 감지/**Slack** 알림 시스템 구축

5. NAS 모니터링 + 용량 자동 정리 스크립트 도입

6. 작업 우선순위 기반 스케줄링 알고리즘(**Priority Scheduling**) 적용

8. 참고 문헌

¹ Thinkbox Software, *Deadline Render Management System — Overview*.

https://docs.thinkboxsoftware.com/products/deadline/10.4/1_User%20Manual/manual/overview.html

² Thinkbox Software, *Deadline Clients, Repository, and Database Architecture*.

https://docs.thinkboxsoftware.com/products/deadline/10.4/1_User%20Manual/manual/overview.html#clients-repository-database

³ Thinkbox Software, *Job and Task Scheduling in Deadline*.

https://docs.thinkboxsoftware.com/products/deadline/10.4/1_User%20Manual/manual/overview.html#jobs-and-tasks

⁴ Thinkbox Software, *Pulse, Web Service, and Additional Components*.

https://docs.thinkboxsoftware.com/products/deadline/10.4/1_User%20Manual/manual/overview.html#pulse-webservice

Thinkbox Software, “Quick Install: Database and Repository,” Deadline User Manual.

https://docs.thinkboxsoftware.com/products/deadline/latest/1_User%20Manual/manual/quick-install-db-repo.html

Combine License Files – Autodesk Official Documentation

<https://www.autodesk.com/support/download-install/admins/network-deploy/combine-license-files>

Borrow a Network License – Autodesk Official Documentation

<https://www.autodesk.com/support/download-install/admins/network-deploy/borrow-a-license>

Deadline Maya Integration Manual Installation – Thinkbox Documentation

https://docs.thinkboxsoftware.com/products/deadline/10.4/1_User%20Manual/manual/app-maya.html#manual-installation